

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

FACULTY OF SCIENCE AND HUMANITIES

DEPARTMENT OF COMPUTER APPLICATIONS



PRACTICAL RECORD NOTE

STUDENT NAME :

REGISTER NUMBER :

CLASS : MCA GEN AI Section : D

YEAR & SEMESTER : I YEAR & II SEMESTER

SUBJECT CODE : PCA25C07J

**SUBJECT TITLE : ARTIFICIAL INTELLIGENCE AND MACHINE
LEARNING**

APRIL 2026



SRM

INSTITUTE OF SCIENCE & TECHNOLOGY
(Deemed to be University u/s 3 of UGC Act, 1956)

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

FACULTY OF SCIENCE AND HUMANITIES

DEPARTMENT OF COMPUTER APPLICATIONS

SRM Nagar, Kattankulathur – 603 203

CERTIFICATE

Certified to be the bonafide record of practical work done by

Register No. _____ of I MCA GEN AI Degree course

for PCA25C07J –ARTIFICIAL INTELLIGENCE AND MACHINE

Learning in the Computer lab in SRM Institute of Science and Technology

during the academic year 2025-2026 .

Staff In-Charge

Head of the Department

Submitted for Semester Practical Examination held on _____.

Internal Examiner

External Examiner

Ex No:1
08/01/2026

Vacuum Cleaner Agent Using Simple Reflex Agent

Aim:

To simulate a simple reflex vacuum cleaner agent that cleans a two-room environment using condition–action rules.

Objectives

- Understand intelligent agent design
- Perform perception → action cycle
- Apply simple decision rules to achieve a goal

Theory:

An **intelligent agent** is a system that:

- Observes the environment (**perception**)
- Takes action to reach a defined **goal**

A **Vacuum Cleaner Agent** follows:

Perception	Action
Room = Dirty	Clean
Room = Clean	Move to next room

Goal → Make **both rooms clean**

This is a **Simple Reflex Agent**.

Problem Description:

A two-room environment consisting of Room A and Room B is given, where each room can be either clean or dirty. A vacuum cleaner agent is placed in one of the rooms and can perform actions such as cleaning the current room or moving to the other room. The problem is to design an agent that uses simple condition–action rules to ensure both rooms are cleaned.

Algorithm:

1. Start
2. Sense current room
3. If Dirty → Clean
4. Else → Move to other room
5. Repeat until all rooms Clean

6. Stop

Source Code:

```
room = {'A': 'Dirty', 'B': 'Clean'}
vacuum_location = 'A'
print('Initial State:', room)
for i in range(2):
    if room[vacuum_location] == 'Dirty':
        print(f'Cleaning Room {vacuum_location}')
        room[vacuum_location] = 'Clean'
    else:
        print(f'Room {vacuum_location} already clean')
    vacuum_location = 'B' if vacuum_location == 'A' else 'A'
print('Final State:', room)
```

Output

```
... Initial State: {'A': 'Dirty', 'B': 'Clean'}
    Cleaning Room A
    Room B already clean
    Final State: {'A': 'Clean', 'B': 'Clean'}
```

Result

The vacuum cleaner agent successfully cleaned the dirty room using simple condition–action rules and achieved the goal of making both rooms clean.

Ex No:2
19/01/2026

Vacuum Cleaner Reflex Agent In Grid Environment

Aim:

To simulate a vacuum cleaner reflex agent using a tile-based grid environment where the agent cleans dirty tiles and avoids obstacles autonomously.

Objectives

- To understand reflex-based decision-making in AI
- To represent real environments using grids
- To apply perception → action cycle for autonomous cleaning
- To implement obstacle-aware movement logic

Theory

An **Intelligent Agent** uses **perception** to act in the environment so that it reaches the desired **goal**.

A **Simple Reflex Agent**:

- Responds only to **current tile condition**
- Uses **Condition** → **Action** rules
- Is fast but **does not remember** past states

Tile Types:

Symbol	Meaning
.	Clean tile
D	Dirty tile
X	Obstacle (cannot enter)

Goal → Make **all reachable dirty tiles clean**

Movement Pattern → **Zig-zag sweep** (like real vacuum robots)

Problem Description

A grid-based environment is provided where each tile can be clean (.), dirty (D), or an obstacle (X). A vacuum cleaner agent starts at the top-left corner and moves in a zig-zag pattern across the grid. The task is to design an agent that cleans all reachable dirty tiles while avoiding obstacles using reflex-based decision rules.

Algorithm

1. Initialize grid with Dirty, Clean, and Obstacle tiles
2. Position the vacuum at tile (0, 0)
3. For each tile in the current row:
 - If row is even: move left → right
 - If row is odd: move right → left
4. If tile is Dirty → Clean
5. If tile is Obstacle → Skip
6. Repeat until all reachable dirty tiles are cleaned
7. Display final floor status

Source Code

. = Clean, D = Dirty, X = Obstacle

```
grid = [  
    [".", ".", ".", ".", "X"],  
    [".", "D", ".", ".", "."],  
    [".", ".", ".", "D", "."]  
]
```

```
rows = len(grid)  
cols = len(grid[0])
```

```
# Vacuum starts at top-left corner  
vacuum_row, vacuum_col = 0, 0
```

```
def print_grid(g):  
    for row in g:  
        print(" ".join(row))  
    print()
```

```
print("Initial Grid:")
```

```

print_grid(grid)
step = 0
# Zig-zag movement: row by row, avoiding obstacles
for r in range(rows):
    if r % 2 == 0:
        col_range = range(cols) # left → right
    else:
        col_range = range(cols - 1, -1, -1) # right → left
    for c in col_range:
        step += 1
        print(f'Step {step}: Moving to ({r}, {c})')
        if grid[r][c] == "X": # Obstacle
            print("- Obstacle here. Skipping this tile.")
            continue
        vacuum_row, vacuum_col = r, c
        if grid[r][c] == "D": # Found dirt
            print("- Tile is Dirty. Cleaning...")
            grid[r][c] = "."
        else:
            print("- Tile is already Clean.")
    print("Current Grid:")
    print_grid(grid)
print("Final Grid:")
print_grid(grid)
print("** All reachable dirty tiles cleaned!")

```

Output:

```

Step 15: Moving to (2, 4)
- Tile is already Clean.
Current Grid:
. . . . X
. . . . .
. . . . .

Final Grid:
. . . . X
. . . . .
. . . . .

** All reachable dirty tiles cleaned!

```

Result

The vacuum cleaner reflex agent successfully navigated the grid environment using a zig-zag movement pattern. It cleaned all reachable dirty tiles and avoided obstacle tiles. The agent completed the cleaning task using perception-action rules without memory

Ex No:3 27/01/2026	Model-Based Traffic Light Controller Agent
------------------------------	---

Aim

To design and implement a Model-Based Intelligent Agent for a traffic light system that makes decisions using both current sensor input and past memory state.

Objectives

- To understand model-based agent architecture
- To implement internal memory for decision making
- To simulate agent behavior in partially observable environments
- To compare reflex and model-based decisions

Theory

A Model-Based Agent is an intelligent agent that maintains internal memory of the environment and uses previous states to make decisions when current input is uncertain.

Working Principle:

Perception → Update Internal Model → Decide Action → Execute

Problem Description

A traffic light system generates signals such as Red (R), Yellow (Y), and Green (G), corresponding to STOP, SLOW, and GO actions. Sometimes, the signal may be invalid or unclear. The problem is to design a model-based agent that uses both current input and stored previous state to make correct decisions even in uncertain conditions.

Algorithm

1. Start
2. Initialize memory (previous_state = None)
3. Read current traffic signal
4. If valid (R/Y/G), update memory
5. Else, use stored memory
6. Apply rules: R→STOP, Y→SLOW, G→GO
7. Print output

8. Repeat

9. Stop

Source Code

```
class TrafficAgent:
    def __init__(self):
        self.previous_state = None

    def decide_action(self, light):
        light = light.upper()

        if light in ["R", "Y", "G"]:
            self.previous_state = light
        else:
            light = self.previous_state
        if light == "R":
            return "STOP - Red Light (stored or sensed)"
        elif light == "Y":
            return "SLOW DOWN - Yellow Light (caution)"
        elif light == "G":
            return "GO - Green Light (proceed)"
        else:
            return "NO ACTION - No previous data available"

agent = TrafficAgent()
signals = ["G", "Y", "BLUR", "R", "UNKNOWN", "G"]
for s in signals:
    print("Signal:", s, "→ Action:", agent.decide_action(s))
```

Output

```
Signal: G → Action: GO - Green Light (proceed)
Signal: Y → Action: SLOW DOWN - Yellow Light (caution)
Signal: BLUR → Action: SLOW DOWN - Yellow Light (caution)
Signal: R → Action: STOP - Red Light (stored or sensed)
Signal: UNKNOWN → Action: STOP - Red Light (stored or sensed)
Signal: G → Action: GO - Green Light (proceed)
```

Result

The model-based agent successfully controlled the traffic signal by using both current sensor input and stored memory of previous states. When invalid signals were received, the agent relied on its internal state to make correct decision

Ex No:4 03/02/2026	Water Jug Problem Using Breadth-First Search
------------------------------	---

Aim

To implement a Problem-Solving Agent for the Water Jug Problem using state-space search to measure an exact target amount of water.

Objectives

- To understand the concept of problem-solving agents in Artificial Intelligence.
- To represent problems using state space representation.
- To apply Breadth-First Search (BFS) to explore possible states.
- To find a sequence of steps to reach the desired goal state.
- To analyze how search algorithms solve real-world problems like the Water Jug Problem

Theory

The Water Jug Problem demonstrates state space representation and BFS search. States are represented as (x, y) where x = water in Jug A (3L) and y = water in Jug B (5L). Initial state: $(0,0)$. Goal: $(*,4)$ or $(4,*)$.

Problem Description

Given two jugs with capacities 3L and 5L, measure exactly 4 liters using allowed operations like filling, emptying, and pouring.

Algorithm

1. Start
2. Define initial state $(0,0)$
3. Add to queue
4. While queue not empty:
 - a. Remove current state
 - b. If goal reached, break
 - c. Generate next states and enqueue
5. Trace back using parent mapping to get path

6. End

Source Code

```
from collections import deque

def water_jug_bfs(jugA, jugB, goal):
    visited = set()
    queue = deque([(0, 0)])
    parent = {(0, 0): None}

    while queue:
        x, y = queue.popleft()

        if x == goal or y == goal:
            path = []
            state = (x, y)

            while state is not None:
                path.append(state)
                state = parent[state]

            return list(reversed(path))

        if (x, y) in visited:
            continue
        visited.add((x, y))
        next_states = set([
            (jugA, y),
            (x, jugB),
            (0, y),
            (x, 0),
            (max(0, x - (jugB - y)), min(jugB, y + x)),
            (min(jugA, x + y), max(0, y - (jugA - x)))
        ])

        for state in next_states:
            if state not in visited:
                parent[state] = (x, y)
                queue.append(state)

jugA, jugB, goal = 3, 5, 4
solution = water_jug_bfs(jugA, jugB, goal)
print("Solution Path:")
for step in solution:
    print(step)
```

Output

```
... Solution Path:  
  (0, 0)  
  (0, 5)  
  (3, 2)  
  (0, 2)  
  (2, 0)  
  (2, 5)  
  (3, 4)
```

Result

The water jug problem was successfully solved using a BFS-based problem-solving agent, which found a sequence of steps to measure exactly 4 liters of water.

Ex No:5

03/02/2026

Uninformed Search Using Breadth-First Search (BFS)

Aim

To implement Breadth-First Search (BFS) as an uninformed search strategy to find the shortest path from a start state to a goal state in a grid-based environment with obstacles.

Objectives

- To understand the concept of uninformed search strategies in Artificial Intelligence.
- To study the working of Breadth-First Search (BFS) algorithm.
- To implement BFS using a queue data structure.
- To find the shortest path from a start node to a goal node in a grid environment.
- To analyze how BFS explores nodes level by level while avoiding obstacles.

Theory

Breadth-First Search (BFS) is an uninformed search algorithm used in Artificial Intelligence for exploring a state space systematically. It expands nodes level by level using a FIFO queue and guarantees the shortest path in an AI problem when step costs are equal.

Problem Description

Given a 5x5 grid where 0 represents a free path and 1 represents an obstacle, the agent must find the shortest path from the start position (0,0) to the goal position (4,4).

Algorithm

1. Start from the initial state.
2. Insert the start node into a queue.
3. Mark the start node as visited.
4. Remove the front node from the queue.
5. Check if it is the goal state.
6. Generate valid neighboring states (up, down, left, right).
7. Add unvisited neighbors to the queue.
8. Repeat until the goal is reached or queue becomes empty.

Source Code

```
from collections import deque
```

```
# 0 = free path, 1 = obstacle
```

```
grid = [  
    [0, 0, 0, 0, 0],  
    [1, 1, 0, 1, 1],
```

```

    [0, 0, 0, 0, 0],
    [0, 1, 1, 1, 0],
    [0, 0, 0, 0, 0]
]

rows = len(grid)
cols = len(grid[0])

directions = [(1,0), (-1,0), (0,1), (0,-1)]

def bfs(start, goal):
    queue = deque([[start]])
    visited = set([start])

    while queue:
        path = queue.popleft()
        r, c = path[-1]

        if (r, c) == goal:
            return path

        for dr, dc in directions:
            nr, nc = r + dr, c + dc

            if 0 <= nr < rows and 0 <= nc < cols:
                if grid[nr][nc] == 0 and (nr, nc) not in visited:
                    visited.add((nr, nc))
                    new_path = list(path)
                    new_path.append((nr, nc))
                    queue.append(new_path)

    return None

start = (0, 0)
goal = (4, 4)
solution = bfs(start, goal)
print("BFS Path:", solution)

```

Output

```
... BFS Path: [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 3), (2, 4), (3, 4), (4, 4)]
```

Result

The BFS algorithm successfully found the shortest path from the start node to the goal node in the grid environment using an uninformed search strategy while avoiding obstacles

Ex No:6
10/02/2026

Uninformed Search Using Depth-First Search (DFS)

Aim

To implement Depth-First Search (DFS) as an uninformed search strategy to find a path from a start state to a goal state in a grid-based maze environment.

Objectives

- To understand the concept of uninformed search techniques.
- To implement Depth-First Search (DFS) using a stack data structure.
- To explore a grid-based environment for pathfinding.
- To identify valid and invalid moves in a maze.
- To find a path from the start state to the goal state.
- To analyze the behavior of DFS in terms of completeness and optimality.

Theory

Depth-First Search (DFS) is an uninformed search algorithm used in Artificial Intelligence to explore a state space deeply before backtracking. It uses a stack (LIFO principle) to expand one branch completely before moving to alternative branches. DFS is memory efficient but does not guarantee the shortest path.

Problem Description

Given a 5x5 grid where 0 represents a free path and 1 represents an obstacle, the agent must find a path from the start position (0,0) to the goal position (4,4) using Depth-First Search.

Algorithm

1. Initialize the stack with the start state.
2. Mark the start state as visited.
3. Repeat until the stack is empty:
4. Pop the top state from the stack.
5. Check if the current state is the goal state.
6. Generate all valid neighboring states (up, down, left, right).
7. Push unvisited neighboring states onto the stack.
8. If goal is reached, return the path.

Source Code

```
# DFS using Stack for Maze Solving
```

```
grid = [  
    [0, 0, 0, 0, 0],  
    [1, 1, 0, 1, 1],  
    [0, 0, 0, 0, 0],  
    [0, 1, 1, 1, 0],  
    [0, 0, 0, 0, 0]  
]
```

```
rows = len(grid)  
cols = len(grid[0])
```

```
directions = [(1,0), (-1,0), (0,1), (0,-1)]
```

```
def dfs(start, goal):  
    stack = [[start]]  
    visited = set([start])  
  
    while stack:  
        path = stack.pop()  
        r, c = path[-1]  
  
        if (r, c) == goal:  
            return path  
  
        for dr, dc in directions:  
            nr, nc = r + dr, c + dc  
  
            if 0 <= nr < rows and 0 <= nc < cols:  
                if grid[nr][nc] == 0 and (nr, nc) not in visited:  
                    visited.add((nr, nc))  
                    new_path = list(path)  
                    new_path.append((nr, nc))  
                    stack.append(new_path)  
  
    return None
```

```
start = (0, 0)  
goal = (4, 4)  
solution = dfs(start, goal)  
print("DFS Path:", solution)
```


Output

```
... DFS Path: [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 1), (2, 0), (3, 0), (4, 0), (4, 1), (4, 2), (4, 3), (4, 4)]
```

Result

The DFS algorithm successfully explored the maze using a stack-based uninformed search strategy and found a valid path from the start state to the goal state

Ex No:7
10/02/2026

Finding Shortest Path Using A* Algorithm

Aim

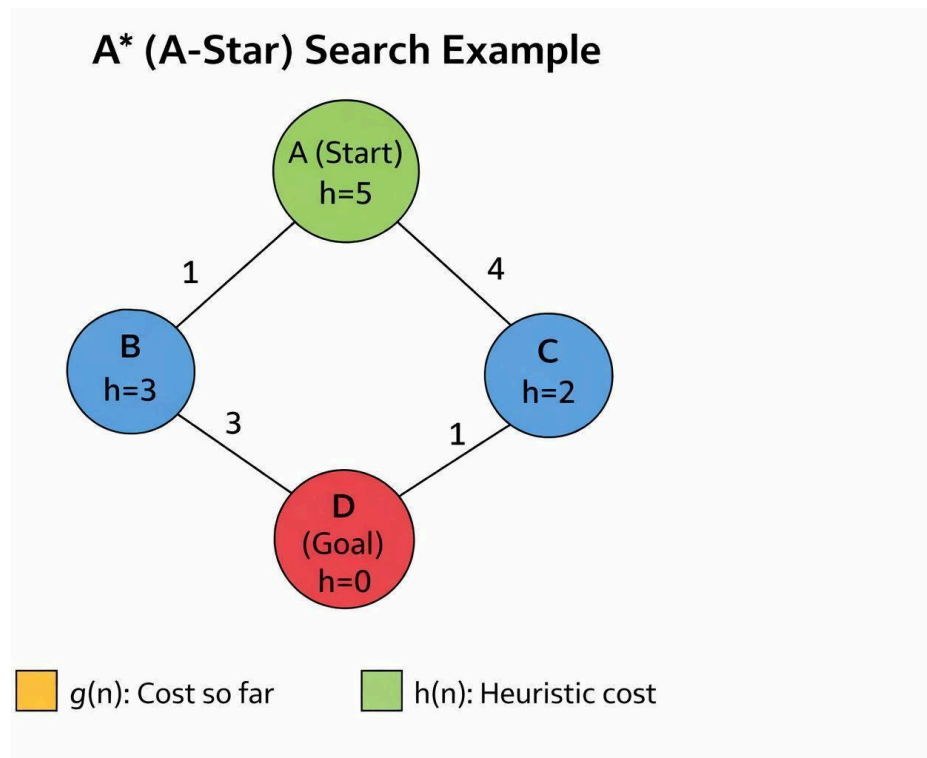
To implement the A* (A-star) search algorithm to find the shortest path between a start node and a goal node in a weighted graph using heuristic values.

Objectives

- To understand informed search strategies.
- To implement A* using the evaluation function $f(n) = g(n) + h(n)$.
- To compute the optimal path from source to destination.

Theory

A* is an informed best-first search algorithm used for pathfinding and graph traversal. It uses two cost functions: $g(n)$, the actual cost from the start node to node n , and $h(n)$, the heuristic estimate from node n to the goal. The total estimated cost is $f(n) = g(n) + h(n)$. The algorithm always expands the node with the lowest $f(n)$ value.



4. Algorithm

1. Initialize open list with the start node.
2. Set $g_cost(start) = 0$ and create parent dictionary.
3. While open list is not empty:
 - Select node with minimum $f(n) = g(n) + h(n)$
 - If it is the goal node, reconstruct the path
 - Otherwise explore its neighbors
 - Update g_cost and parent if a better path is found
 - Add neighbor to open list if not already present
4. Return the shortest path

Source Code:

```
def a_star(graph, heuristic, start, goal):
    open_list = [start]
    g_cost = {start: 0}
    parent = {}

    while open_list:
        current = min(open_list, key=lambda node: g_cost[node] + heuristic[node])
        open_list.remove(current)

        if current == goal:
            path = []
            while current in parent:
                path.append(current)
                current = parent[current]
            path.append(start)
            path.reverse()
            return path

        for neighbor, cost in graph[current]:
            new_cost = g_cost[current] + cost

            if neighbor not in g_cost or new_cost < g_cost[neighbor]:
                g_cost[neighbor] = new_cost
                parent[neighbor] = current
```

```
        if neighbor not in open_list:
            open_list.append(neighbor)

    return None

graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('D', 3)],
    'C': [('D', 1)],
    'D': []
}

heuristic = {
    'A': 5,
    'B': 3,
    'C': 2,
    'D': 0
}

start = 'A'
goal = 'D'

path = a_star(graph, heuristic, start, goal)

print("Shortest Path:", path)
```

Output

```
Shortest Path: ['A', 'B', 'D']
```

Result

The A* algorithm was successfully implemented and the optimal path from A to D was obtained using heuristic-based informed search

Ex No:8
17/02/2026

Medical Diagnosis Expert System Using Prolog

Aim

To design and implement a simple rule-based Medical Diagnosis Expert System using Prolog to identify diseases based on symptoms.

Objectives

- To understand rule-based programming in Prolog.
- To implement facts, rules, and dynamic predicates.
- To demonstrate unification and backtracking concepts.
- To simulate a simple expert system.

Theory

An Expert System is an Artificial Intelligence application that uses knowledge and inference rules to solve problems in a specific domain. Prolog is a logic programming language based on First Order Predicate Logic. It uses facts, rules, and queries to derive conclusions.

This medical expert system diagnoses diseases based on user-provided symptoms. It uses dynamic predicates to store responses and logical rules to infer diseases.

Problem Description

In real-world scenarios, diagnosing diseases requires analyzing multiple symptoms and matching them with medical knowledge. This problem focuses on building a simple expert system that takes user input in the form of symptoms and identifies the possible disease using predefined rules. The system simulates basic medical reasoning using Prolog's logical inference capabilities.

Algorithm

1. Start the program.
2. Ask the user about symptoms.
3. Store user responses dynamically.
4. Match symptoms with predefined disease rules.
5. Display the diagnosed disease.
6. Clear stored responses.

Source Code:

`:- dynamic symptom/1.`

start :-

```
write('-----'), nl,  
write(' MEDICAL DIAGNOSIS EXPERT SYSTEM '), nl,  
write('-----'), nl,  
nl,  
diagnose(Disease),  
write('You may have: '), write(Disease), nl,  
nl,  
write('Note: This is only a basic expert system.'), nl,  
write('Please consult a doctor for proper diagnosis.'), nl,  
undo.
```

diagnose(fever) :-

```
symptom(high_temperature),  
symptom(body_pain),  
symptom(headache).
```

diagnose(common_cold) :-

```
symptom(runny_nose),  
symptom(sneezing),  
symptom(sore_throat).
```

diagnose(malaria) :-

```
symptom(high_temperature),  
symptom(chills),  
symptom(sweating).
```

diagnose(typhoid) :-

```
symptom(high_temperature),  
symptom(weakness),  
symptom(stomach_pain).
```

diagnose(covid19) :-

```
symptom(fever),  
symptom(dry_cough),  
symptom(loss_of_taste).
```

symptom(Symptom) :-

```
( known(yes, Symptom)
```

```

-> true
; known(no, Symptom)
-> fail
; ask(Symptom)
).
ask(Symptom) :-
    write('Do you have '), write(Symptom), write('? (yes/no): '),
    read(Response),
    nl,
    asserta(known(Response, Symptom)),
    Response = yes.
undo :-
    retract(known(_, _)),
    fail.
undo.

```

Output

```

-----
MEDICAL DIAGNOSIS EXPERT SYSTEM
-----

Do you have high_temperature? (yes/no): yes.

Do you have body_pain? (yes/no): |: yes.

Do you have headache? (yes/no): |: yes.

You may have: fever

Note: This is only a basic expert system.
Please consult a doctor for proper diagnosis.
true .

```

Result

Thus, the Medical Diagnosis Expert System was successfully implemented using Prolog and tested with sample inputs.

Ex No:9 24/02/2026	Student Performance Evaluation Expert System Using Prolog
------------------------------	--

Aim

To design and implement a rule-based expert system using Prolog to evaluate student performance based on academic inputs.

Objectives

- To understand expert system concepts using Prolog.
- To apply facts and rules for decision-making.
- To evaluate student performance based on predefined criteria.
- To demonstrate logical inference and backtracking.

Theory

An Expert System simulates human decision-making using rules and knowledge. This system evaluates student performance using attendance, marks, and participation.

Problem Description

The problem is to develop an expert system that evaluates a student's overall performance based on inputs such as marks, attendance, and participation. The system should ask relevant questions, process responses using predefined rules, and classify the student into categories like excellent, good, average, or poor.

Algorithm

1. Start the program.
2. Ask the user about student details.
3. Store responses dynamically.
4. Apply rules to evaluate performance.
5. Display performance result.
6. Clear stored data.

Source Code:

`:- dynamic known/2.`

`start :-`

`write('-----'), nl,`

`write(' STUDENT PERFORMANCE EXPERT SYSTEM '), nl,`


```
write('-----'), nl,  
nl,  
evaluate(Result),  
write('Student Performance: '), write(Result), nl,  
undo.
```

```
evaluate(excellent) :-  
response(high_marks),  
response(high_attendance),  
response(active_participation).
```

```
evaluate(good) :-  
response(moderate_marks),  
response(high_attendance).
```

```
evaluate(average) :-  
response(moderate_marks),  
response(low_attendance).
```

```
evaluate(poor) :-  
response(low_marks).
```

```
response(Q) :-  
(known(yes, Q) -> true ;  
known(no, Q) -> fail ;  
ask(Q)).
```

```
ask(Q) :-  
write('Does the student have '), write(Q), write('? (yes/no): '),  
read(R), nl,  
assert(known(R, Q)),  
R = yes.
```

```
undo :- retract(known(_, _)), fail.  
undo.
```

Output

```
-----  
STUDENT PERFORMANCE EXPERT SYSTEM  
-----  
  
Does the student have high_marks? (yes/no): yes.  
Does the student have high_attendance? (yes/no): |: yes.  
Does the student have active_participation? (yes/no): |: yes.  
  
Student Performance: excellent  
true .  
  
23 ?- ▲
```

Result

Thus, the Student Performance Expert System was successfully implemented

Ex No:10
24/02/2026

Animal Identification Expert System Using Prolog

Aim

To develop an expert system using Prolog to identify animals based on their characteristics.

Objectives

- To understand rule-based classification.
- To implement knowledge representation in Prolog.
- To identify animals using logical rules.
- To demonstrate inference mechanisms.

Theory

Animal identification systems use observable characteristics such as habitat, body type, and behavior to classify animals.

Problem Description

The problem is to build an expert system that identifies an animal based on its characteristics such as habitat, physical features, and abilities. The system should interact with the user, collect inputs, apply logical rules, and infer the correct animal.

Algorithm

1. Start the program.
2. Ask about animal characteristics.
3. Store responses.
4. Match rules.
5. Display result.
6. Clear responses.

Source Code:

`:- dynamic known/2.`

```
start :-  
write('-----'), nl,  
write(' ANIMAL IDENTIFICATION SYSTEM '), nl,  
write('-----'), nl,
```

```
nl,  
identify(Animal),  
write('The animal is: '), write(Animal), nl,  
undo.
```

```
identify(tiger) :-  
feature(carnivore),  
feature(has_stripes).
```

```
identify(elephant) :-  
feature(herbivore),  
feature(has_trunk).
```

```
identify(eagle) :-  
feature(can_fly),  
feature(has_sharp_vision).
```

```
identify(fish) :-  
feature(lives_in_water),  
feature(has_fins).
```

```
identify(frog) :-  
feature(lives_in_water),  
feature(can_jump).
```

```
feature(F) :-  
(known(yes, F) -> true ;  
known(no, F) -> fail ;  
ask(F)).
```

```
ask(F) :-  
write('Does the animal have '), write(F), write('? (yes/no): '),  
read(R), nl,  
asserta(known(R, F)),  
R = yes.
```

```
undo :- retract(known(_, _)), fail.  
Undo.
```

Output

```
-----  
ANIMAL IDENTIFICATION SYSTEM  
-----  
  
Does the animal have carnivore? (yes/no): yes.  
Does the animal have has_stripes? (yes/no): |: yes.  
  
The animal is: tiger  
true .
```

Result

The animal identification expert system successfully identified animals based on user-provided characteristics using logical rules.

Ex No:11
03/03/2026

Student Score Prediction Using Linear Regression

Aim

To implement Linear Regression using Python to predict test scores based on hours studied.

Objectives

- To understand the concept of Linear Regression.
- To use scikit-learn for building a regression model.
- To analyze the relationship between study time and test scores.
- To visualize the regression line using Matplotlib.

Theory

Linear Regression is a supervised learning technique used to model the relationship between an independent variable and a dependent variable. It follows the equation $y = mx + c$, where y is the dependent variable, x is the independent variable, m is the slope, and c is the intercept. The model finds the best-fitting straight line by minimizing prediction errors.

Problem Description

The problem is to predict a student's test score based on the number of hours studied. Using a given dataset, a linear regression model is trained to understand the relationship and predict scores accordingly.

Algorithm

1. Import required libraries.
2. Define input data (hours studied and test scores).
3. Reshape the input data.
4. Create a Linear Regression model.
5. Train the model.
6. Predict scores.
7. Display coefficient and intercept.
8. Plot the graph.

Source Code:

```
import numpy as np
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt
```

```
hours_studied = np.array([1, 2, 3, 4, 5]).reshape(-1, 1)
test_scores = np.array([50, 55, 65, 70, 75])

model = LinearRegression()
model.fit(hours_studied, test_scores)

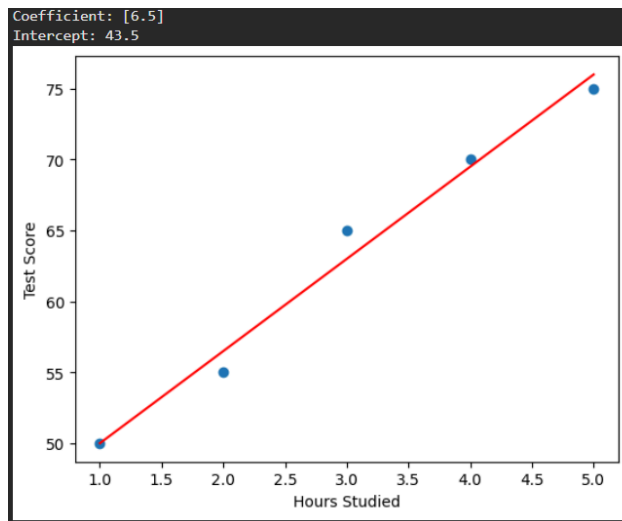
predicted_scores = model.predict(hours_studied)

print("Coefficient:", model.coef_)
print("Intercept:", model.intercept_)

plt.scatter(hours_studied, test_scores)
plt.plot(hours_studied, predicted_scores, color='red')

plt.xlabel('Hours Studied')
plt.ylabel('Test Score')
plt.show()
```

Output



Result

Thus, the Linear Regression model was successfully implemented and used to predict test scores based on hours studied.

Ex No:12
03/03/2026

Iris Flower Classification Using Logistic Regression

Aim

To implement Logistic Regression using Python to classify iris flower species based on input features.

Objectives

- To understand the concept of Logistic Regression.
- To build a classification model using Python.
- To classify data into multiple categories.
- To evaluate model performance using accuracy and classification report.

Theory

Logistic Regression is a supervised machine learning algorithm used for classification problems. It predicts the probability of a categorical dependent variable using a logistic (sigmoid) function. The output is mapped to discrete classes. In multi-class classification, Logistic Regression uses techniques such as one-vs-rest to classify data into more than two categories.

Problem Description

The problem is to develop a classification model using Logistic Regression to identify the species of iris flowers based on their input features such as sepal length, sepal width, petal length, and petal width. The dataset contains multiple classes (setosa, versicolor, and virginica), and the model must learn the relationship between the features and the target labels. The objective is to train the model on a given dataset, test it on unseen data, and accurately classify the iris species while evaluating its performance using appropriate metrics.

Algorithm

1. Import required libraries.
2. Load the dataset from a CSV file.
3. Convert categorical labels into numerical values.
4. Separate features and target variables.
5. Split the dataset into training and testing sets.
6. Create a Logistic Regression model.
7. Train the model using training data.
8. Predict results on test data.
9. Evaluate the model using accuracy and classification report.

Source Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

# Load dataset
df = pd.read_csv("iris.csv")

# Encode labels
df['species'] = df['species'].map({
    'setosa': 0,
    'versicolor': 1,
    'virginica': 2
})

# Features and target
X = df.iloc[:, 0:2].values
y = df.iloc[:, -1].values

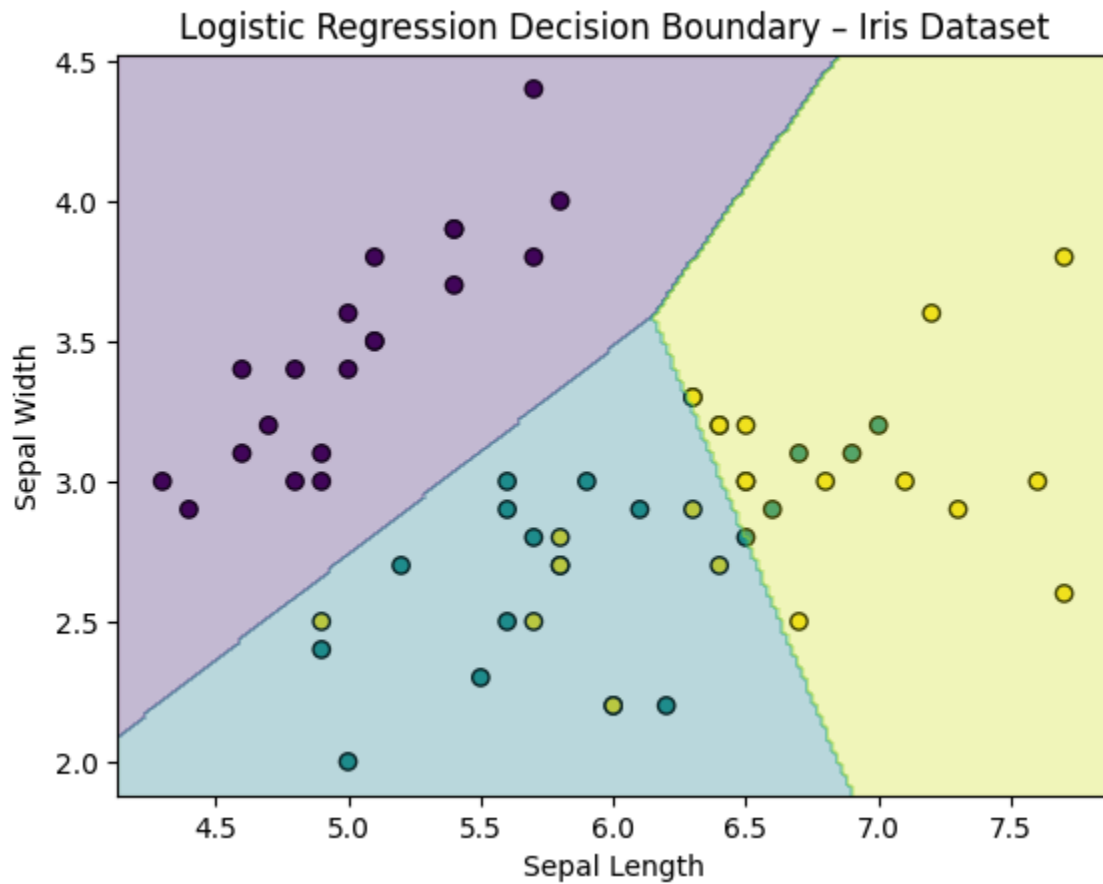
# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train model
model = LogisticRegression(max_iter=200)
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Evaluate
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

Output



Accuracy: 0.8333333333333334

	precision	recall	f1-score	support
0	1.00	1.00	1.00	4
1	0.50	1.00	0.67	2
2	1.00	0.67	0.80	6
accuracy			0.83	12
macro avg	0.83	0.89	0.82	12
weighted avg	0.92	0.83	0.84	12

Result

Thus, the Logistic Regression model was successfully implemented to classify iris flower species, and the performance was evaluated using accuracy and classification metrics

Ex No:13
10/03/2026

Customer Purchase Prediction Using Decision Tree

Aim

To implement a Decision Tree classifier using Python to predict whether a customer will buy a computer.

Objectives

- To understand the concept of Decision Trees.
- To perform classification using Decision Tree algorithm.
- To convert categorical data into numerical form.
- To evaluate model performance using accuracy.
- To visualize the decision tree.

Theory

Decision Tree is a supervised machine learning algorithm used for classification and regression. It works by splitting the dataset into subsets based on the most significant feature, forming a tree-like structure. Each internal node represents a decision, each branch represents an outcome, and each leaf node represents a class label.

Problem Description

The problem is to build a model that predicts whether a customer will buy a computer based on features such as age, income, student status, and credit rating. The dataset contains categorical values which must be converted into numerical form before training the model.

Algorithm

1. Import required libraries.
2. Load the dataset from CSV file.
3. Convert categorical values into numerical values.
4. Define features and target variable.
5. Split dataset into training and testing sets.
6. Create Decision Tree classifier.
7. Train the model using training data.
8. Predict output using test data.
9. Calculate accuracy of the model.
10. Visualize the decision tree.

Source Code:

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn import tree
import matplotlib.pyplot as plt

# Load dataset
df = pd.read_csv('buy_computer.csv')

# Convert categorical data to numerical
df['Age'] = df['Age'].map({'<=30': 1, '31...40': 2, '>40': 3})
df['Income'] = df['Income'].map({'Low': 1, 'Medium': 2, 'High': 3})
df['Student'] = df['Student'].map({'No': 0, 'Yes': 1})
df['CreditRating'] = df['CreditRating'].map({'Fair': 1, 'Excellent': 2})
df['BuysComputer'] = df['BuysComputer'].map({'No': 0, 'Yes': 1})

# Features and target
X = df[['Age', 'Income', 'Student', 'CreditRating']]
y = df['BuysComputer']

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

# Create model
clf = DecisionTreeClassifier()

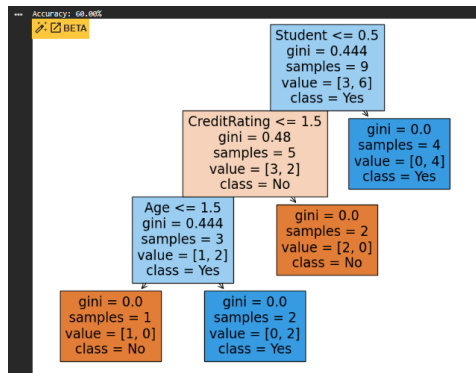
# Train model
clf.fit(X_train, y_train)

# Predict
y_pred = clf.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# Plot tree
plt.figure(figsize=(10,8))
tree.plot_tree(clf, filled=True,
               feature_names=['Age','Income','Student','CreditRating'],
               class_names=['No','Yes'])
plt.show()
```

Output



Result

Thus, the Decision Tree classifier was successfully implemented and visualized for predicting whether a customer will buy a computer.

Ex No:14
17/03/2026

Iris Flower Classification Using Support Vector Machine

Aim

To implement a Support Vector Machine (SVM) classifier using Python to classify the Iris dataset.

Objectives

- To understand the concept of Support Vector Machines.
- To perform classification using SVM.
- To split dataset into training and testing sets.
- To evaluate model performance using accuracy and classification report.

Theory

Support Vector Machine (SVM) is a supervised machine learning algorithm used for classification and regression tasks. It works by finding the optimal hyperplane that separates data points of different classes with maximum margin. In this experiment, SVM is applied to the Iris dataset to classify different species of flowers based on their features.

Problem Description

The problem is to build a machine learning model that classifies iris flowers into different species using their features. The system should load the dataset, train an SVM model, test it on unseen data, and evaluate its performance.

Algorithm

1. Import required libraries.
2. Load the Iris dataset.
3. Select input features and target labels.
4. Split dataset into training and testing sets.
5. Create SVM classifier model.
6. Train the model using training data.
7. Predict output using test data.
8. Evaluate performance using accuracy and classification report.

Source Code:

```
# Import libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets
```

```

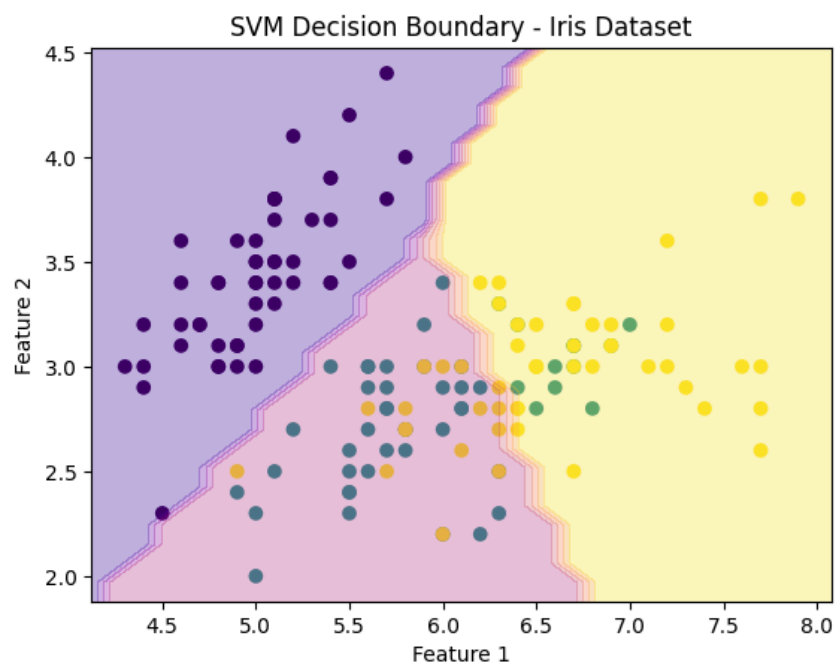
from sklearn import svm
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

# Load dataset
iris = datasets.load_iris()
X = iris.data[:, :2] # first two features
y = iris.target
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# Create SVM model
model = svm.SVC(kernel='linear')

# Train model
model.fit(X_train, y_train)
# Predict
y_pred = model.predict(X_test)
# Evaluate
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))

```

Output



```

... Accuracy: 0.9
      precision    recall  f1-score   support

     0         1.00      1.00      1.00        10
     1         0.88      0.78      0.82         9
     2         0.83      0.91      0.87        11

 accuracy          0.90
 macro avg          0.90      0.90      0.90
 weighted avg       0.90      0.90      0.90

```

Result

Thus, the Support Vector Machine classifier was successfully implemented and evaluated using the Iris dataset.

Ex No:15
17/03/2026

Customer Segmentation Using K-Means Clustering

Aim

To implement K-Means Clustering using Python to group customers based on income and spending score.

Objectives

- To understand the concept of clustering.
- To implement K-Means clustering using Python.
- To group similar data points into clusters.
- To visualize clusters using a graph.

Theory

K-Means Clustering is an unsupervised machine learning algorithm used to group similar data points into clusters. It works by dividing the dataset into k clusters, where each data point belongs to the cluster with the nearest mean (centroid). The algorithm iteratively updates cluster centroids until convergence is achieved. It is widely used for customer segmentation and pattern recognition.

Problem Description

The problem is to group customers based on their annual income and spending score. Using a small dataset, the K-Means algorithm is applied to divide customers into clusters such that customers in the same group have similar characteristics.

Algorithm

1. Import required libraries.
2. Create or load the dataset.
3. Select relevant features.
4. Apply K-Means clustering algorithm.
5. Assign cluster labels to data points.
6. Print clustered data.
7. Plot clusters and centroids.

Source Code:

```
# Import libraries
import pandas as pd          # For reading dataset
import matplotlib.pyplot as plt  # For plotting graphs
from sklearn.cluster import KMeans  # K-Means clustering algorithm
```

```

from sklearn.metrics import silhouette_score

df = pd.read_csv("customer_kmeans_dataset.csv") # Load the dataset

# Select features for clustering
# We use Income and Spending Score
X = df[['Annual_Income_k$', 'Spending_Score_1_100']].values

wcss = [] # WCSS = Within Cluster Sum of Squares (measure of cluster error)

# Test different numbers of clusters
for i in range(1, 11): # Try cluster numbers from 1 to 10

    kmeans = KMeans(n_clusters=i, random_state=42) # Create KMeans model

    kmeans.fit(X) # Train the model using dataset

    wcss.append(kmeans.inertia_) # Store WCSS value (cluster error)

# Plot Elbow Graph
plt.plot(range(1, 11), wcss) # X-axis = number of clusters, Y-axis = WCSS
plt.title("Elbow Method for Optimal K") # Graph title
plt.xlabel("Number of Clusters (K)") # X-axis label
plt.ylabel("WCSS") # Y-axis label
plt.show() # Display graph

# Assume elbow graph suggests K = 5
kmeans = KMeans(n_clusters=4, random_state=42)

y_kmeans = kmeans.fit_predict(X) # Train model and assign clusters

score = silhouette_score(X, y_kmeans) # Calculate silhouette score

print("Silhouette Score:", score) # Print clustering quality

# Plot clustered data
plt.scatter(X[:,0], X[:,1], c=y_kmeans) # Color points by cluster

# Plot centroids (cluster centers)
plt.scatter(kmeans.cluster_centers_[0],
            kmeans.cluster_centers_[1],

```

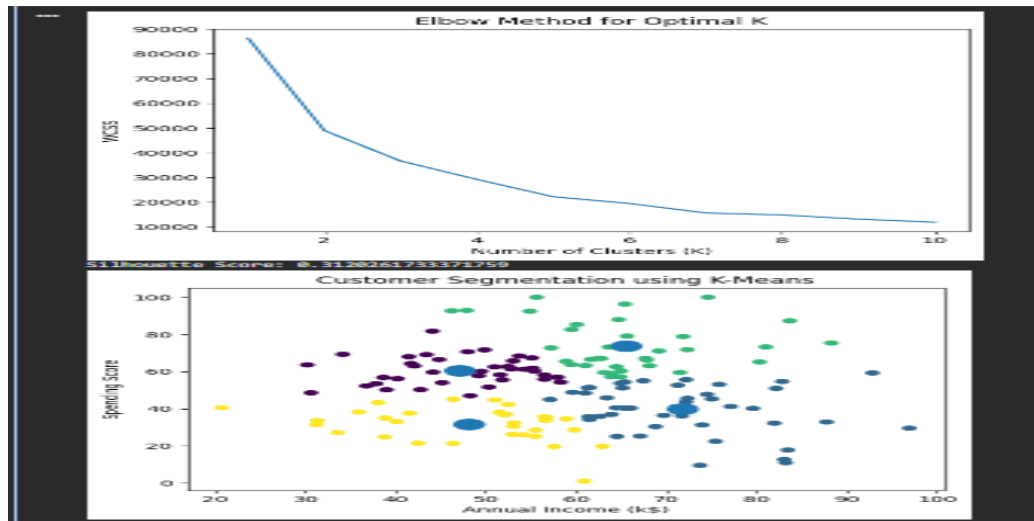
```

s=200)                                # Larger size for visibility

plt.title("Customer Segmentation using K-Means")
plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score")
plt.show()

```

Output



Result

Thus, the K-Means clustering algorithm was successfully implemented to group customers into clusters based on their income and spending score.